

МІНІСТЕРСТВО ОСВІТИ І НАУКИ УКРАЇНИ
Національний університет «Запорізька політехніка»

Кафедра програмних засобів
(найменування кафедри)

КУРСОВИЙ ПРОЄКТ
(РОБОТА)

з «Безпека та захист програм і даних»
(назва дисципліни)
на тему: «Застосунок шифрування-дешифрування алгоритмом Віженера»
(назва теми)

Студента(ки) IV курсу КНТ-122 групи
спеціальності 121«Інженерія програмного забезпечення»
освітня програма (спеціалізація) бакалавр
Онищенко О.А.
(ПРИЗВИЩЕ та ініціали)

Керівник: Качан О.І.
(посада, вчене звання, науковий ступінь, ПРИЗВИЩЕ та ініціали)

Кількість балів: _____

Члени комісії:	_____	<u>Тетяна ЗАЙКО</u>
	(підпис)	(Ім'я ПРИЗВИЩЕ)
	_____	<u>Олександр КАЧАН</u>
	(підпис)	(Ім'я ПРИЗВИЩЕ)
	_____	_____
	(підпис)	(Ім'я ПРИЗВИЩЕ)

МІНІСТЕРСТВО ОСВІТИ І НАУКИ УКРАЇНИ
Національний університет «Запорізька політехніка»

Факультет комп'ютерних наук і технологій
Кафедра «Програмні засоби»
Ступінь вищої освіти бакалавр
Спеціальність 121 «Інженерія програмного забезпечення»
(код і найменування)
Освітня програма (спеціалізація) _____
(назва освітньої програми (спеціалізації))

З А В Д А Н Н Я
НА КУРСОВИЙ ПРОЄКТ (РОБОТУ) СТУДЕНТА(КИ)

ОНИЩЕНКА Олега Антоновича

(ПРІЗВИЩЕ, ім'я, по батькові)

1. Тема проєкту (роботи) «Застосунок шифрування-дешифрування алгоритмом Віженера»
керівник роботи доцент, КАЧАН Олександр Іванович
(науковий ступінь, вчене звання, ПРІЗВИЩЕ, ім'я, по батькові)
2. Строк подання студентом проєкту (роботи) 25 грудня 2025 р.
3. Вихідні дані до проєкту (роботи) технічне завдання
4. Зміст розрахунково-пояснювальної записки (перелік питань, які потрібно розробити) 1 АНАЛІЗ ПРЕДМЕТНОЇ ОБЛАСТІ. 2 ПРОЄКТУВАННЯ СТРУКТУРИ СИСТЕМИ. 3 ПРОГРАМНА РЕАЛІЗАЦІЯ. 4 РОЗРОБКА ІНТЕРФЕЙСУ. 5 КЕРІВНИЦТВО КОРИСТУВАЧА

КАЛЕНДАРНИЙ ПЛАН

№ з/п	Назва етапів курсового проєкту (роботи)	Строк виконання етапів проєкту (роботи)	Примітка
1	Отримання завдання	1 год	
2	Розробка технічного завдання	3 год	
3	Розробка програми	12 год	
4	Заповнення пояснювальної записки	3 год	

Студент(ка)

(підпис)

Олег ОНИЩЕНКО

(Ім'я ПРІЗВИЩЕ)

Керівник проєкту (роботи)

(підпис)

Олександр КАЧАН

(Ім'я ПРІЗВИЩЕ)

РЕФЕРАТ

Пояснювальна записка до курсового проекту містить: 44 стор., 10 рис., 4 табл., 4 додатк., 5 джрл.

ВІЖЕНЕР, ВЕБ ЗАСТОСУНОК, ВІДКРИТИЙ КОД, ШИФРУВАННЯ, ДЕШИФРУВАННЯ, FLASK, NEXTJS.

Об'єкт проєктування — сучасні системи шифрування та дешифрування тексту.

Предмет проєктування — застосунок шифрування та дешифрування тексту шифром Віженера.

Мета проєкту — створити застосунок, що зможе шифрувати та дешифрувати текст алгоритмом Віженера.

Методи дослідження – використання відкритих джерел інформації та відомих засобів розробки.

Результатом роботи має бути готова програма, пояснювальна записка, презентація проєкту.

Фінальна версія програми має включати такі функції:

- шифрування;
- дешифрування;
- копіювання вмісту з буфера обміну.

Застосунок може бути використаний на таких платформах:

- Windows;
- MacOS;
- Android;
- iOS.

Фінальна версія програми має бути розроблена з використанням фреймворку Flask на серверній частина програми та фреймворку Next.js на клієнтській.

ЗМІСТ

Реферат.....	3
Перелік умовних позначень, скорочень, термінів.....	6
Вступ.....	7
1 Аналіз предметної області та огляд аналогів.....	8
1.1 Аналіз предметної області.....	8
1.2 Огляд існуючих методів шифрування та дешифрування.....	8
1.2.1 Веб-застосунок CaesarCipher.....	8
1.2.2 Веб-застосунок Vortex.....	9
1.3 Висновки за першим розділом.....	10
2 Проектування структури системи.....	11
2.1 Створення структурної схеми.....	11
2.2 Створення функціональної схеми.....	11
3 Програмна реалізація.....	12
3.1 Вибір засобів розробки.....	12
3.1.1 Обґрунтування вибору засобів розробки серверної частини.....	12
3.1.2 Обґрунтування вибору засобів розробки клієнтської частини.....	13
3.1.3 Обґрунтування вибору середовища розробки.....	14
3.2 Висновки за розділом.....	14
4 Розробка інтерфейсу.....	15
4.1 Вибір платформи.....	15
4.2 Розробка макету інтерфейсу.....	15
4.3 Вигляд розробленого інтерфейсу.....	16
4.4 Висновок за розділом.....	16
5 Керівництво користувача.....	17
5.1 Необхідні умови для виконання.....	17
5.2 Звертання до програми.....	17
Висновки.....	19
Перелік джерел посилань.....	20
Додаток А.....	21
Додаток Б.....	26
Додаток В.....	31

ПЕРЕЛІК УМОВНИХ ПОЗНАЧЕНЬ, СКОРОЧЕНЬ, ТЕРМІНІВ

Flask	бібліотека розробки серверної частини застосунку
JavaScript	мова програмування веб-застосунків
NextJS	фреймворк розробки клієнтської частини застосунку
Python	високорівнева мова програмування
TailwindCSS	сучасна бібліотека стилізації веб-сторінок

ВСТУП

В сучасному світі існує багато інформації. Цю інформацію часто пересилають друзям або зберігають на віддалених сховищах. Під час перенесення інформації з одного місця в інше є ризик її викрадення, пошкодження або втрати.

Для запобігання втрати інформації її зазвичай передають у зміненому вигляді. Таку інформацію зашифровано різними шифрами.

Існує багато шифрів. Наприклад, шифр Полібія, шифр Цезаря та шифр Віженера. Шифр Віженера відрізняється тим, що він має одне ключове слово, яким кодуються символи. Шифр Віженера є варіацією шифра заміни.

Після шифрування користувач отримує шифрований текст та залишає ключове слово. Для розшифрування потрібно використати таблицю Віженера, але у зворотному порядку.

1 АНАЛІЗ ПРЕДМЕТНОЇ ОБЛАСТІ ТА ОГЛЯД АНАЛОГІВ

1.1 Аналіз предметної області

Шифрування — це перетворення тексту на такий, що його буде складно прочитати. Це є техніка приховування даних. Зазвичай шифрування передбачає наявність ключа, через який працює сам алгоритм.

Шифрування поділяють на два види: симетричне і несиметричне. Симетричне шифрування використовує тільки один ключ як для шифрування, так і для дешифрування. Асиметричне шифрування використовує два ключі — публічний та приватний. [1]

1.2 Огляд існуючих методів шифрування та дешифрування

Існують такі шифри:

- шифр простої заміни;
- шифр Полібія;
- шифр Цезаря;
- шифр Тритемія;
- шифр Віженера.

Шифр Віженера відрізняється тим, що має такий ключ, який повторюється. На основі ключа обирається наступний символ. Цей шифр є поліалфавітним, тобто він використовує кілька алфавітів для заміни символів. [2]

1.2.1 Веб-застосунок CaesarCipher

Веб-застосунок Caesar Cipher дозволяє шифрувати та дешифрувати текст багатьма алгоритмами. Один з алгоритмів є шифр Віженера. На сторінці веб-застосунку можна ввести текст або завантажити текст з файлу.

Після вказання ключа цей веб-застосунок шифрує або дешифрує наданий текст.

Переваги:

- простий інтерфейс;
- можливість шифрувати текст різних регістрів;
- можливість шифрувати та дешифрувати числа.

Рисунок 1.1 — Вигляд основної частини веб-застосунку Caesar Cipher

1.2.2 Веб-застосунок Vovxentriq

Веб-застосунок Vovxentriq також дозволяє шифрувати та дешифрувати текст шифром Віженера. Цей застосунок надає можливість використання різних мов. Застосунок також дозволяє копіювати текст з буфера обміну.

Переваги:

- можливість розшифрування тексту;
- можливість використання різних мов;
- дозволяє використовувати символи, що не входять до алфавіту.

Vigenère Cipher

 Language

English

Note: Autosolver supports fewer languages.

Manual/autosolver mode

 **Manual**
Encrypt / decrypt

 **Autosolver**
Crack ciphertext

Enter text + key. Choose Encrypt or Decrypt.

Ciphertext

Tools

Lcu Mcg yc oujhj hkk krxzg, zvdz Vh moyk Vly Cqrm Ekurzhht Grt, hkgh zncvusykf ekzlkhh ob Koa vncxrr quh skflyv eah kgjh kjhxzdyhltu ooth. (Pckt 3:16)

Mode

Decrypt

Choose Encrypt = Plaintext → Ciphertext or Decrypt = Ciphertext → Plaintext

Key

GOD

Enter the key used for encryption or decryption.

Letters outside A–Z

Keep as is

Determines if letters are converted to their Latin equivalents (like Å → AA, Æ → AE).

Plaintext

Tools

For God so loved the world, that He gave His Only Begotten Son, that whosoever beliieth in Him should not perish but have everlasting life. (John :)

► Explanation

Рисунок 1.2 — Вигляд основної частини веб-застосунку Voxentriq

1.3 Висновки за першим розділом

Обидва застосунки реалізують шифр Віженера коректно. Застосунок Caesar Cipher містить зручніший та більш мінімалістичний інтерфейс. Застосунок Voxentriq має більший набір функцій.

2 ПРОЄКТУВАННЯ СТРУКТУРИ СИСТЕМИ

2.1 Створення структурної схеми

Розроблений веб-застосунок складається з двох частин:

- клієнтська, розроблена з фреймворком Next.js;
- серверна, розроблена з бібліотекою Flask.

Структура програми описана на структурній схемі, що на рисунку нижче:

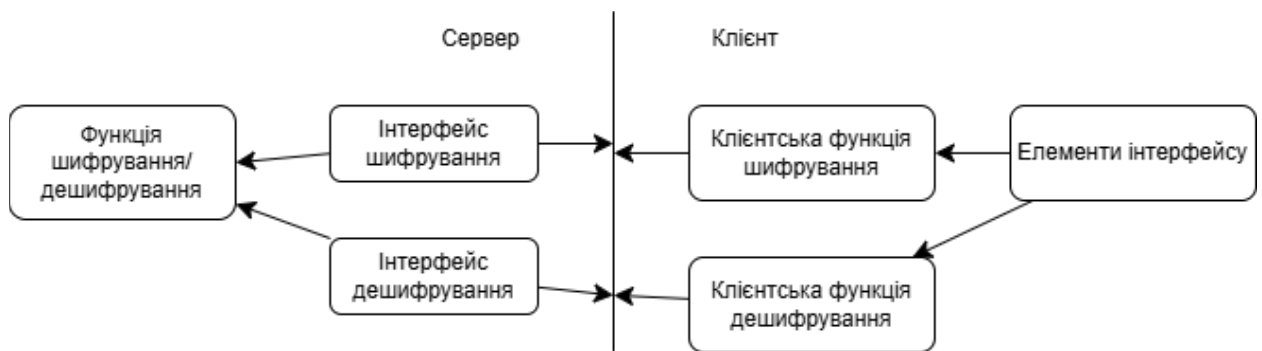


Рисунок 2.1 — Структурна схема застосунку

2.2 Створення функціональної схеми

Розроблений веб-застосунок включає такі функції:

- шифрування тексту;
- дешифрування тексту;
- копіювання тексту;
- вставлення тексту з буфера обміну.

Функціональна схема описана на рисунку нижче:

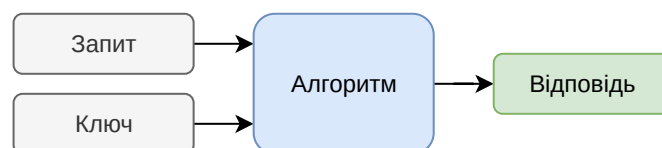


Рисунок 2.2 — Функціональна схема веб-застосунку

3 ПРОГРАМНА РЕАЛІЗАЦІЯ

3.1 Вибір засобів розробки

Для розробки серверної частини обрано мову програмування Python та бібліотеку Flask.

Для розробки клієнтської частини обрано мову програмування JavaScript, бібліотеку React та фреймворк Next.js. Для стилізації клієнтської частини обрано бібліотеку TailwindCSS.

3.1.1 Обґрунтування вибору засобів розробки серверної частини

Мову програмування Python обрано через її легкий синтаксис, наявність великої кількості бібліотек та широку підтримку спільноти.

Порівняння мови програмування Python з іншими опціями наведено нижче на таблиці:

Таблиця 3.1 — Порівняння мов програмування

Критерій	Python	Java	C#
Типізація	Динамічна	Статична	Статична
Наявність бібліотек	Висока	Середня	Середня
Підтримка спільноти	Висока	Середня	Середня

Фреймворк для мови Python обрано Flask. Він використовується для створення серверних точок доступу. Основними перевагами фреймворку є його простота застосування та ясність документації.

Порівняння фреймворку Flask з іншими опціями наведено на таблиці нижче:

Таблиця 3.2 — Порівняння фреймворків розробки серверу

Критерій	Flask	Django	FastAPI
Простота використання	Висока	Середня	Висока
Підтримка спільноти	Середня	Висока	Середня
Простота документації	Висока	Середня	Висока

3.1.2 Обґрунтування вибору засобів розробки клієнтської частини

Мову програмування JavaScript обрано через її універсальне використання для веб-розробки.

Бібліотеку React обрано через її популярність, простоту написання та підтримку спільноти.

Порівняння інших можливих бібліотек розробки наведено нижче на таблиці:

Таблиця 3.3 — Порівняння бібліотек для клієнтської частини

Критерій	React	Vue	Svelte
Простота розробки	Середня	Висока	Середня
Підтримка спільноти	Висока	Середня	Низька
Наявність додаткових модулів	Висока	Середня	Низька

Фреймворк Next.js обрано через його широку підтримку спільнотою, простоту налаштування та легку можливість розгортання на платформі Vercel.

Порівняння його з іншими фреймворками наведено нижче на таблиці:

Таблиця 3.4 — Порівняння фреймворків для бібліотеки React

Критерій	Next.js	Remix	TanStack
Простота розгортання	Висока	Середня	Низька
Підтримка спільноти	Висока	Середня	Низька
Підтримка додаткових модулів	Висока	Середня	Середня

3.1.3 Обґрунтування вибору середовища розробки

Середовище розробки VS Code обрано через його популярність, легкість налаштування та підтримку спільноти.

Інші варіанти порівняно на таблиці нижче:

Таблиця 3.5 — Порівняння інтегрованих середовищ розробки

Критерій	Visual Studio Code	WebStorm
Простота	Висока	Середня
Кількість налаштувань	Середня	Висока
Підтримка спільноти	Висока	Середня

3.2 Висновки за розділом

Обрані засоби розробки обґрунтовано та порівняно з їхніми аналогами.

4 РОЗРОБКА ІНТЕРФЕЙСУ

4.1 Вибір платформи

Як платформу для веб-застосунку обрано веб-браузер через його поширеність та можливість розробки одразу і під мобільні пристрої.

4.2 Розробка макету інтерфейсу

Попередній вигляд інтерфейсу наведено на рисунку нижче:

The diagram illustrates the layout of the web application interface. It consists of the following elements arranged vertically:

- A large white rounded rectangle labeled "Текст" (Text).
- A smaller white rounded rectangle labeled "Ключ" (Key).
- Two light blue rounded rectangles side-by-side: "Шифрувати" (Encrypt) on the left and "Дешифрувати" (Decrypt) on the right.
- A large white rounded rectangle labeled "Шифротекст" (Ciphertext).
- A single light blue rounded rectangle labeled "Копіювати" (Copy) positioned below the "Шифротекст" box.

Рисунок 4.1 — Вигляд інтерфейсу веб-застосунку

4.3 Вигляд розробленого інтерфейсу

Vigenere Cipher

Paste

Given text

For example, Some text...

Key

For example, GOD

Output will be here...

Reverse Copy

*For God so loved the world
that He gave His only
begotten Son, that whoever
believes in Him should not
perish, but have eternal life.
(John 3:16)*

N

Рисунок 4.1 — Вигляд розробленого веб-інтерфейсу

4.4 Висновок за розділом

Було розроблено схему інтерфейсу та сам інтерфейс із раніше обраними засобами розробки.

5 КЕРІВНИЦТВО КОРИСТУВАЧА

5.1 Необхідні умови для виконання

Для виконання програми потрібно:

1. завантажити архів програми собі на систему;
2. розархівувати код програми;
3. завантажити та встановити Python останньої версії.

Аби запустити програму потрібно:

1. з папки коду ввести команду *npm run dev*;
2. далі ввести до консолі *.venv\scripts\activate.py, python main.py*;
3. перейти до сторінки <http://localhost:3000/>.

5.2 Звертання до програми

Програма складається з однієї сторінки та трьох частин цієї сторінки.

Перша частина є заголовком. На ньому написана назва програми з посиланням на інформаційну сторінку. Вигляд першої частини наведено нижче на рисунку:



Рисунок 5.1 — Вигляд заголовку програми

Друга частина є основною і вона містить функціонал програми. Згори кнопка Paste дозволяє вставити текст з буфера обміну до поля вводу. Далі йде основне поле вводу тексту для шифрування чи дешифрування. Після нього поле вводу ключа. Ще далі поле виводу результату. Під ним дві кнопки — перша Reverse надсилає зашифрований текст до поля вводу, а друга Copy копіює вміст поля виведення до буфера обміну.

Вигляд основної частини програми наведено нижче на рисунку:

Рисунок 5.2 — Вигляд основної частини програми

Третьою частиною є Біблійний вірш, який також дозволяє його скопіювати до буфера обміну і вставити до поля шифрування як приклад.

Вигляд третьої частини застосунку наведено на рисунку нижче:

Рисунок 5.3 — Вигляд третьої частини застосунку

ВИСНОВКИ

Благодаттю Господа нашого Ісуса Христа після виконання роботи було проведено ознайомлення із предметною областю – шифрування за допомогою шифру Віженера. Також було розроблено веб-застосунок на основі цієї предметної області, який дозволяє шифрувати та дешифрувати текст алгоритмом Віженера.

У першому розділі було оглянуто принцип роботи програм шифрування шифром Віженера. Також було розглянуто особливості програм прикладів та їх інтерфейси.

У другому розділі було розглянуто структуру програми. Було визначено структурну та функціональну схему застосунку.

У третьому розділі було обрано засоби розробки. Вибір було обґрунтовано і порівняно з іншими опціями.

У четвертому розділі було розроблено інтерфейс веб-застосунку. Було не тільки зроблено схему інтерфейсу, а й наведено вигляд готового інтерфейсу.

У п'ятому розділі було розглянуто кожен елемент інтерфейсу окремо з його функціоналом.

Надалі покращити програму можна розмістивши код на віддаленому сервері та додаванням можливості роботи з файлами.

ПЕРЕЛІК ДЖЕРЕЛ ПОСИЛАНЬ

1. [Що таке шифрування і як воно працює- IT News](https://it-news.com.ua/statti/shyfruvannia-yak-zakhystyty-svoi-dani-u-tsyfrovu-epokhu.html) [Electronic resource]. – Access mode: <https://it-news.com.ua/statti/shyfruvannia-yak-zakhystyty-svoi-dani-u-tsyfrovu-epokhu.html>
2. [Vigenère cipher - Wikipedia](https://en.wikipedia.org/wiki/Vigen%C3%A8re_cipher) [Electronic resource]. – Access mode: https://en.wikipedia.org/wiki/Vigen%C3%A8re_cipher

ДОДАТОК А
Технічне завдання

A.1 Вступ

A.1.1 Назва програми

Програма має назву «Застосунок шифрування-дешифрування алгоритмом Віженера».

A.1.2 Призначення та область застосування

Програма призначена для шифрування та дешифрування тексту шифром Віженера.

Вона може бути використана звичайними людьми, колегами по роботі, тощо. Така програма спрощує зашифровану комунікацію або приватні документи.

A.2 Підстави для розробки

Підставою розробки програми є створення застосунку, що має полегшити шифрування та дешифрування тексту в різних ситуаціях.

A.3 Призначення розробки

Програма має дозволяти користувачам вводити власний текст разом з ключем, а на виході отримувати зашифровані дані.

Це може бути корисним при передачі закритої інформації.

A.4 Вимоги до програми

Основними вимогами до програми є:

- зручний графічний користувацький інтерфейс;
- використання шифру Віженера.

А.4.1 Вимоги до функціональних характеристик

Програма має виконувати такі мінімальні функції:

- введення тексту для шифрування;
- введення ключу шифрування;
- виведення результатів користувачеві.

А.4.2 Вимоги до надійності

Програма має містити обробники виключних ситуацій аби не допустити непланового завершення роботи.

Для надійності даних вони можуть зберігатися у буфері обміну.

А.4.3 Вимоги до технічних та програмних засобів

Програма має запускатися у браузері і бути доступною як на мобільних, так і на домашніх комп'ютерах.

Мінімальні вимоги до клієнтської частини:

- процесор: x64 / x86 / ARM;
- оперативна пам'ять: 512 МБ;
- накопичувач: 1 ГБ вільного місця;
- операційна система: Android, iOS, macOS, Linux, Windows.

Програмні засоби можуть змінюватись в ході розробки, але заплановано використати такі:

- Flask (бекенд фреймворк для Python);
- TailwindCSS (класовий фреймворк CSS).

Для користувацького інтерфейсу програма має використовувати фреймворк Next.js. TailwindCSS має бути використаний для стилізації.

A.4.4 Вимоги до вхідних даних

Вимогами до ключа та введеного тексту є підтримка повної таблиці ASCII, а не лише літер символів алфавіту.

A.5 Вимоги до програмної документації

Програмна документація має бути реалізована у:

- пояснювальній записці;
- технічному завданні.

Код програми має бути написаний зі зрозумілими назвами змінних/класів/методів і може включати коментарі.

A.6 Стадії і етапи розробки

A.6.1 Стадії розробки

Стадії розробки:

1. розробка документації;
2. написання програми;
3. розробка презентації.

A.6.2 Етапи розробки

Етапи розробки:

1. вибір технічних засобів;
2. ітеративне написання програми;
3. тестування та перероблювання;
4. заповнення пояснювальної записки.

А.6.3 Зміст робіт по етапах

Зміст роботи може бути таким на кожному етапі:

1. початкова документація, тестові програми;
2. робоча програма, більш заповнена документація, підготовка до презентації;
3. готова програма, готова документація, готова презентація.

ДОДАТОК Б
Текст програми

Код серверної частини — vigenere.py

```

from flask import Flask, request
from flask_cors import CORS

app = Flask(__name__)
CORS(app)
app.config["CORS_HEADERS"] = "Content-Type"
ASCII_LENGTH = 128

class Mode:
    ENCRYPT = "encrypt"
    DECRYPT = "decrypt"

@app.route("/encrypt/")
def encrypt():
    query = request.args.to_dict()
    result = vigenere(query["text"], query["key"], Mode.ENCRYPT)
    return result

@app.route("/decrypt/")
def decrypt():
    query = request.args.to_dict()
    result = vigenere(query["text"], query["key"], Mode.DECRYPT)
    return result

def vigenere(message: str, key: str, mode: str = "encrypt") -> str:
    # Make key as long as the message is
    key_stream = key * len(message)
    # Truncate it, so that it does not exceed the message length
    long_key = key_stream[: len(message)]

    result: str = str()
    for index, _ in enumerate(message):
        key_ascii = ord(long_key[index])
        message_ascii = ord(message[index])

        letter_ascii = 0
        if mode == Mode.ENCRYPT:
            letter_ascii = (message_ascii + key_ascii) %
ASCII_LENGTH
        elif mode == Mode.DECRYPT:
            letter_ascii = (message_ascii - key_ascii) %
ASCII_LENGTH
        else:
            raise ValueError("Invalid mode...")

        new_letter = chr(letter_ascii)
        result += new_letter

    return result

if __name__ == "__main__":
    app.run(host="0.0.0.0", port=501000, debug=True)

```

Текст сторінки — page.tsx

```

"use client";

import { useState } from "react";

export default function Home() {
  const [text, setText] = useState("");
  const [key, setKey] = useState("");
  const [output, setOutput] = useState("");

  const baseUrl = "http://127.0.0.1:42248";
  const handleEncrypt = async () => {
    const url = `${baseUrl}/encrypt/?text=${text}&key=${key}`;
    const result = await (await fetch(url)).text();
    setOutput(result);
  };
  const handleDecrypt = async () => {
    const url = `${baseUrl}/decrypt/?text=${text}&key=${key}`;
    const result = await (await fetch(url)).text();
    setOutput(result);
    console.log(key);
  };
  const handlePaste = async () => {
    const data = await navigator.clipboard.readText();
    setText(data);
  };

  const copyResult = () => {
    setText(output);
  };

  const INPUT_CLASSES =
    "outline-2 rounded-md outline-sky-300 focus:outline-sky-500
px-2";
  const BUTTON_CLASSES =
    "bg-sky-50 p-1 hover:bg-sky-200 w-full rounded-md cursor-
pointer";
  const BIBLE_VERSE =
    "For God so loved the world that He gave His only begotten Son,
that whoever believes in Him should not perish, but have eternal life.";

  return (
    <div className="min-h-screen bg-sky-50 flex gap-3 flex-col
items-center justify-center">
      {/* Header */}
      <div className="rounded-md bg-white border-2 border-sky-300
flex p-3 gap-3 w-60">
        <span>🔒</span>
        <h1>
          <a
            href="https://en.wikipedia.org/wiki/Vigenère_cipher"
            className="underline underline-offset-2"
          >
            Vigenere Cipher
          </a>
        </h1>
      </div>
    </div>
  )
}

```

```

    { /* Main App */ }
    <div className="rounded-md bg-white border-2 border-sky-300
flex flex-col p-3 gap-4">
      <button className={BUTTON_CLASSES} onClick={handlePaste}>
        Paste
      </button>
      <div className="flex flex-col gap-1">
        <label htmlFor="givenText">Given text</label>
        <textarea
          name="inputText"
          id="inputText"
          className={` ${INPUT_CLASSES} resize-none`}
          rows={3}
          placeholder="For example, Some text..."
          value={text}
          onChange={ (e) => setText(e.target.value)}
        ></textarea>
      </div>

      <div className="flex flex-col gap-1">
        <label htmlFor="keyInput">Key</label>
        <input
          type="text"
          id="keyInput"
          className={INPUT_CLASSES}
          placeholder="For example, GOD"
          value={key}
          onChange={ (e) => {
            const keyValue = e.target.value;
            const upperKey = keyValue.toUpperCase();
            const keyWithNoSymbols = upperKey.replace(/[^a-zA-
Z]/g, "");
            setKey(keyWithNoSymbols);
          }}
        />
      </div>

      <div className="flex flex-row gap-3" hidden={!text || !key}
>
        <button className={BUTTON_CLASSES}
onClick={handleEncrypt}>
          Encrypt
        </button>
        <button className={BUTTON_CLASSES}
onClick={handleDecrypt}>
          Decrypt
        </button>
      </div>

      <div className="flex flex-col gap-2">
        <textarea
          name="ciphethOutput"
          id="ciphethOutput"
          className={` ${INPUT_CLASSES} resize-none`}
          rows={3}
          placeholder="Output will be here..."
          value={output}
          readOnly
          onClick={ () => navigator.clipboard.writeText(output)}
        ></textarea>
        <div className="flex flex-row justify-end gap-2">

```

```

px-2"
    <button
      className="bg-sky-50 rounded-md self-end w-fit py-1
    >
      Reverse
    </button>
    <button
      className="bg-sky-50 rounded-md self-end w-fit py-1
    >
      Copy
    </button>
  </div>
</div>
</div>

{ /* Bible */
  <div className="rounded-md bg-white border-2 border-sky-300
flex flex-col p-3 w-60">
  <span
    className="italic text-justify cursor-copy"
    onClick={ () =>
navigator.clipboard.writeText (BIBLE_VERSE) }
  >
    {BIBLE_VERSE}
  </span>
  <span className="self-end">
    (
      <a
        className="underline underline-offset-2"
        href="https://www.biblegateway.com/passage/?
search=John%203%3A16&version=UKR"
      >
        John 3:16
      </a>
    )
  </span>
</div>
</div>
);
}

```

ДОДАТОК В

Слайди презентації

Шифр Віженера

Курсова робота,
Безпека та захист програм і даних

викладач, Качан О.І.
студент КНТ-122, Онищенко О.А.

Рисунок В.1 - Слайд

Зміст

01

Презентація містить такі розділи:

1. Порівняння фреймворків
2. Порівняння середовищ розробки
3. Розбір алгоритму
4. Порівняння прикладів
5. Приклади використання

Рисунок В.2 - Слайд

Порівняння фреймворків

02

Критерій	Django	Nuxt	NiceGUI
Мова	Python	TypeScript	Python
Вид	бекенд	фул-стак	фронтенд (змішаний)
Підтримка	+	++	-
Простота	-	+	++
Наявність додаткових модулів	+	++	-

Рисунок В.3 - Слайд

Порівняння середовищ розробки

03

Критерій	Visual Studio Code	Notepad	neovim
Зрозумілість інтерфейсу	+	++	-
Легкість освоєння	+	++	-
Зручність	+	+	+
Додаткові плагіни	++	-	+
Підтримка спільноти	+++	-	++

Рисунок В.4 - Слайд

Розбір алгоритму

04

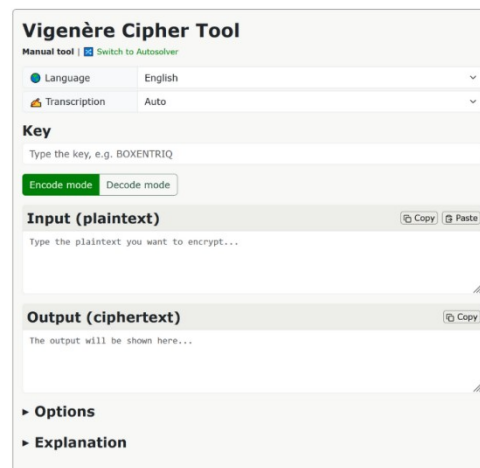
Кроки виконання алгоритму Віженера:

1. Отримати літеру ключа
2. Підбрати літеру з таблиці
3. Повторювати для кожної літери

Рисунок В.5 - Слайд

Приклад застосунку: Boxentriq

05



The image shows a web-based application titled "Vigenère Cipher Tool". It has a toggle switch between "Manual tool" and "Switch to Autosolver". Below this are two dropdown menus: "Language" set to "English" and "Transcription" set to "Auto". There is a "Key" input field with the placeholder text "Type the key, e.g. BOXENTRIQ". Below the key field are two buttons: "Encode mode" (highlighted in green) and "Decode mode". There is an "Input (plaintext)" section with a text area and a "Copy" button. Below that is an "Output (ciphertext)" section with a text area and a "Copy" button. At the bottom, there are two expandable sections: "Options" and "Explanation".

Рисунок В.6 - Слайд

Приклад застосунку: dCode

06



Рисунок В.7 - Слайд

Виконання застосунку

07



Рисунок В.8 - Слайд

Висновки**08**

Так бо Бог полюбив світ, що дав
Сина Свого Однородженого, щоб
кожен, хто вірує в Нього, не
згинув, але мав життя вічне.

Івана 3:16

Рисунок В.9 - Слайд